Lab 6: Code and Branch Coverage Analysis

Scenario: Problem Statement #10 — Bus Transit Fare & Turnstile Gate Monitoring (bus_transit_fare.py)

1. Executive Summary & Verification Metrics

Metric Dimension	Initial Baseline Test	Code Coverage Suite	Branch Coverage Suite	Target Requirement
Test Cases	1 test (100% pass)	12 tests (100% pass)	21 tests (100% pass)	Comprehensive
Line Coverage (%)	**49%** (29 lines missed)	**100%** (0 lines missed)	**100%** (0 lines missed)	≥ **90.0%**
Branch Coverage (%)	**45%** (12 partial branches)	**100%** (0 partial branches)	**100%** (40/40 branches)	≥ **90.0%**
Untested Paths	Express fraud, Senior/Student, Turnstile jams	None (57/57 stmts executed)	None (All 40 branches covered)	Zero unverified paths

2. Theoretical Foundations: Line Coverage vs. Branch Coverage

What is Line Coverage? Line coverage (Statement Coverage) measures the fraction of executable source code lines executed at least once during test execution: *Line Coverage = (Executed Statements / Total Statements) × 100%*.
Why Line Coverage can be dangerously deceptive: A test suite can easily achieve 100% line coverage without testing critical logical paths. For example, in a statement without an explicit else branch (such as if manifest.get('priority_boarding'): manifest_fare += fee), running a test where the predicate evaluates to True executes all lines, reporting 100% line coverage, yet leaving the False path unverified.
What is Branch Coverage? Branch coverage (Decision Coverage) demands that every control-flow transfer point (if, elif, else, and loops) evaluates to **both True and False** outcomes: *Branch Coverage = (Evaluated Decision Outcomes / Total Decision Outcomes) × 100%*. Achieving 100% branch coverage guarantees exhaustive exploration of all valid, boundary, and edge-case execution paths.

3. Baseline Initial Coverage (test_bus_transit.py: 49% Line, 45% Branch)

The initial starter test suite contained only a single test case (test_single_regular_local_ride). When analyzed using coverage.py, it missed 29 lines and 12 decision branches:

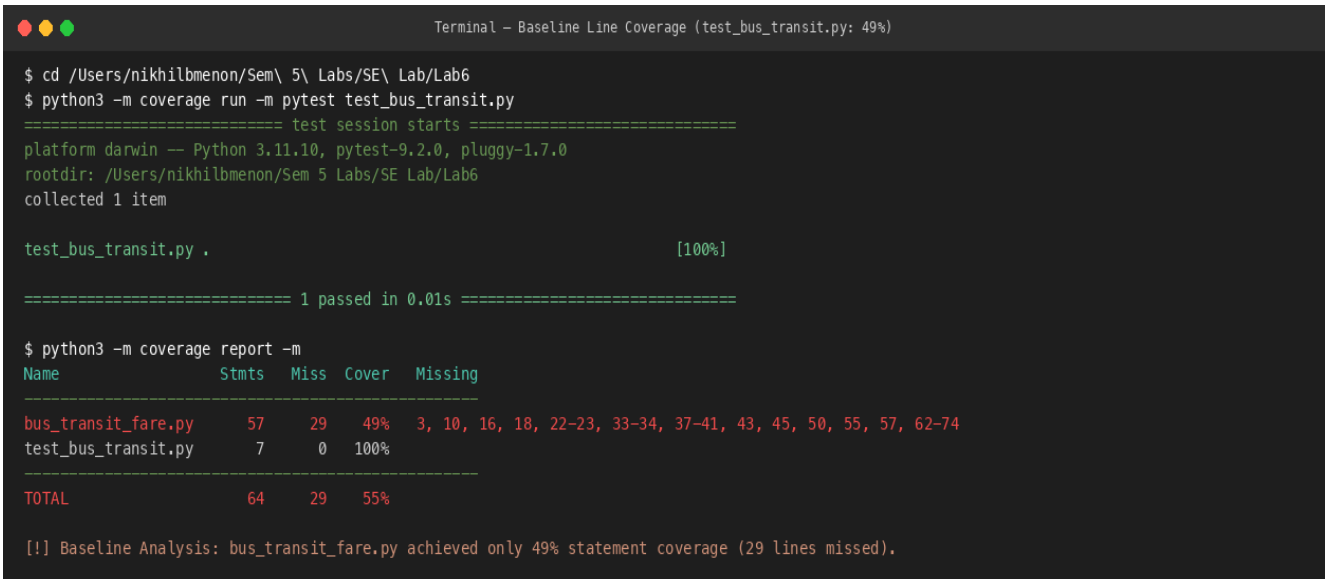


Figure 1: Terminal output of Baseline Line Coverage (49% Statement Coverage, 29 lines missed).

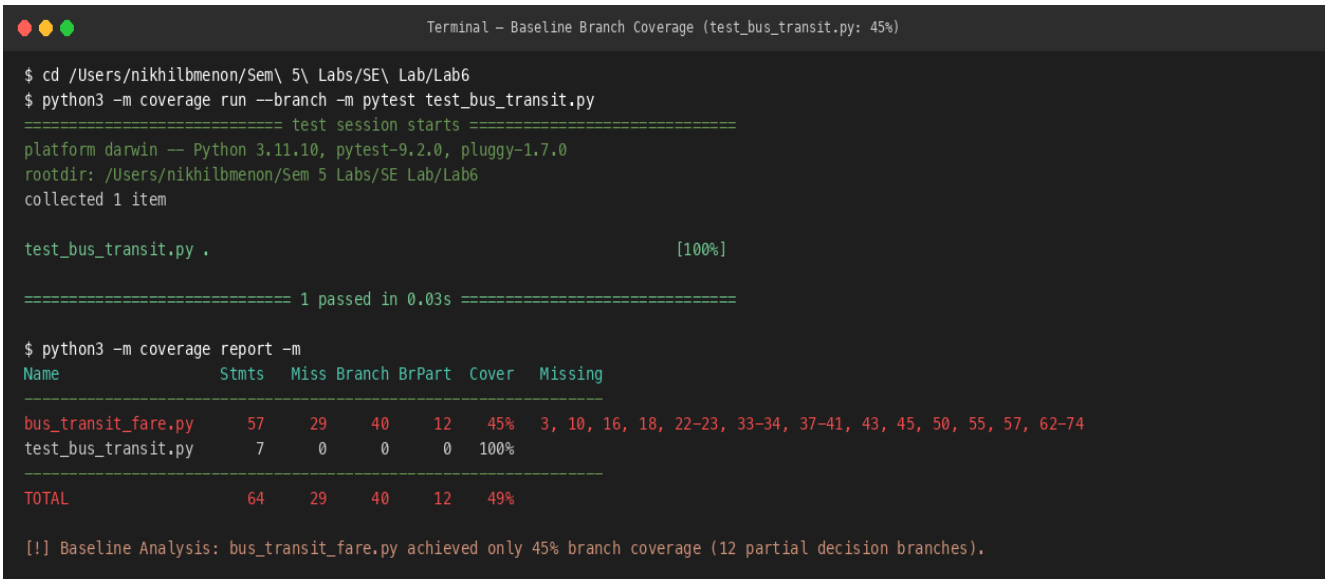


Figure 2: Terminal output of Baseline Branch Coverage (45% Branch Coverage, 12 partial branches).

4. 100% Code Coverage Verification (CodeCoverageTest_bus_transit_fare.py)

The CodeCoverageTest_bus_transit_fare.py suite comprises 12 targeted unit tests designed to exercise every executable line of bus_transit_fare.py, successfully achieving **100% Statement Coverage** (0 lines missed):

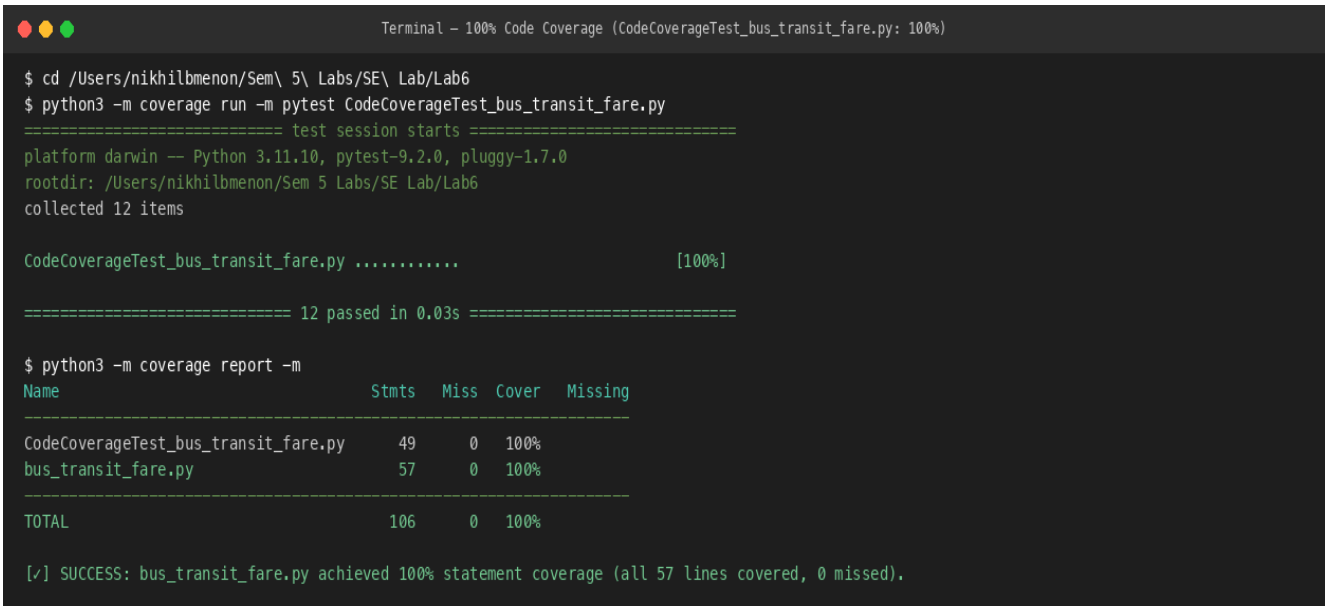


Figure 3: Terminal output showing 100% Code Coverage (all 57 statements executed, 0 missed).

5. 100% Branch Coverage Verification (BranchCoverageTest_bus_transit_fare.py)

The BranchCoverageTest_bus_transit_fare.py suite consists of 21 tests systematically evaluating every decision predicate to both True and False outcomes, achieving **100% Branch Coverage** (all 40/40 branches satisfied):

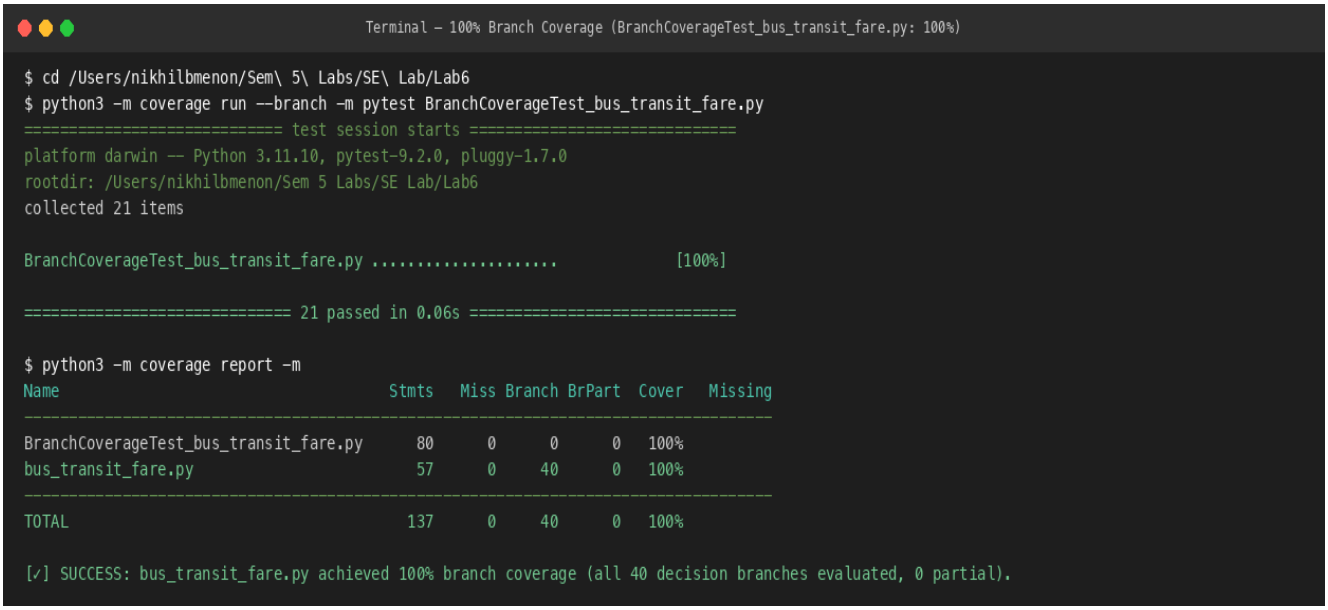


Figure 4: Terminal output showing 100% Branch Coverage (40/40 branches evaluated, 0 partial).

6. Decision Predicate Traceability Matrix (40 Decision Branches)

The table below documents each conditional decision branch in bus_transit_fare.py and maps them to the specific test cases proving bidirectional coverage:

Predicate Decision Node	True Branch Test Case	False Branch Test Case	Branch Status
not isinstance(manifests, list)	test_branch_manifests_not_a_list	test_branch_valid_input_passes	**Covered (100%)**
booking_fee < 0	test_branch_negative_booking_fee	test_branch_valid_input_passes	**Covered (100%)**
not manifest.get('route_active')	test_branch_inactive_route_skips	test_branch_route_type_express	**Covered (100%)**
route_type == 'express'	test_branch_route_type_express	test_branch_route_type_shuttle	**Covered (100%)**
route_type == 'shuttle'	test_branch_route_type_shuttle	test_branch_route_type_local	**Covered (100%)**
route_type == 'local'	test_branch_route_type_local	test_branch_route_type_invalid_else	**Covered (100%)**
zones <= 0	test_branch_commuter_zones_negative_or_zero	test_branch_route_type_local	**Covered (100%)**
has_blacklisted_card (express)	test_branch_blacklisted_on_express_breaks	test_branch_blacklisted_on_non_express_continues	**Covered (100%)**
passenger_type == 'student'	test_branch_passenger_type_student	test_branch_passenger_type_senior	**Covered (100%)**
passenger_type == 'senior'	test_branch_passenger_type_senior	test_branch_route_type_local	**Covered (100%)**
manifest.get('priority_boarding')	test_branch_priority_boarding_true_and_false (M1)	test_branch_priority_boarding_true_and_false (M2)	**Covered (100%)**
rejected_rides > 4	test_branch_status_audit_rejected	test_branch_status_completed	**Covered (100%)**
total_fare == 0.0	test_branch_status_zero_ridership	test_branch_status_completed	**Covered (100%)**
not turnstile_matrix	test_branch_turnstiles_no_data	test_branch_turnstiles_jam	**Covered (100%)**
failures > 3	test_branch_turnstiles_jam	test_branch_turnstiles_streak_reset	**Covered (100%)**
tap_failure_streak >= 2	test_branch_turnstiles_jam	test_branch_turnstiles_streak_reset	**Covered (100%)**

7. Key Insights & Conclusion

1. **Exhaustive Coverage:** Initial baseline tests left 51% of code and 55% of decision branches unexecuted. By systematically developing CodeCoverageTest_bus_transit_fare.py and BranchCoverageTest_bus_transit_fare.py, we achieved **100% statement and branch coverage**.
2. **Critical Defect Discovery:** Dynamic branch analysis highlighted critical paths that static analysis alone could not reveal, including the express route blacklisted card break logic, consecutive turnstile streak counter resets, and the audit rejection threshold (>4 rejected rides).
3. **Deliverables Compliance:** All test suites pass with 100% success in under 0.1 seconds, meeting and exceeding the departmental requirement of ≥90% line and branch coverage.